Machine Learning Assignment :- 11

Name: Ghrushnesh Sudhakar Patil

Class: CSAI-D

Batch : 3

Roll No: 66

PRN: 12411658

Q. 11. Assignment on Classification technique. Every year many students give the GRE exam to get admission in foreign Universities. The data set contains GRE Scores (out of 340), TOEFL Scores (out of 120), University Rating (out of 5), Statement of Purpose strength (out of 5), Letter of Recommendation strength (out of 5), Undergraduate GPA (out of 10), Research Experience (0=no, 1=yes), Admitted (0=no, 1=yes). Admitted is the target variable.

DATASET LINK :- https://www.kaggle.com/mohansacharya/graduate-admissions

The counsellor of the firm is supposed check whether the student will get an admission or not based on his/her GRE score and Academic Score. So to help the counsellor to make appropriate decisions, build a machine learning model classifier using a Decision tree to predict whether a student will get admission or not. Apply Data pre-processing (Label Encoding, Data Transformation....) techniques if necessary.

1. Perform data-preparation (Train-Test Split)

2. Apply Machine Learning Algorithm

3. Evaluate Model.

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
```

```
# Loaded dataset
df = pd.read_csv("Admission_Predict_Ver1.1.csv")
df.shape
```

```
(500, 9)
```

```
# Removeing extra spaces from column names
df.columns = df.columns.str.strip()
df.columns
```

```
Index(['Serial No.', 'GRE Score', 'TOEFL Score', 'University Rating', 'SOP',
       'LOR', 'CGPA', 'Research', 'Chance of Admit', 'Admitted'],
      dtype='object')
```

```
# Display first 5 rows
print("First 5 rows of dataset:")
print(df.head())
```

```
First 5 rows of dataset:
   Serial No.  GRE Score  TOEFL Score  University Rating  SOP  LOR   CGPA  \
0           1        337          118                  4  4.5   4.5  9.65
1           2        324          107                  4  4.0   4.5  8.87
2           3        316          104                  3  3.0   3.5  8.00
3           4        322          110                  3  3.5   2.5  8.67
4           5        314          103                  2  2.0   3.0  8.21

   Research  Chance of Admit
0         1             0.92
1         1             0.76
2         1             0.72
3         1             0.80
4         0             0.65
```

```
# Checked missing values
print("\nMissing values:")
print(df.isnull().sum())
```

```
Missing values:
Serial No.           0
GRE Score            0
TOEFL Score          0
University Rating    0
SOP                  0
LOR                  0
CGPA                 0
Research             0
Chance of Admit      0
dtype: int64
```

```
# Create binary target variable from Chance of Admit
df['Admitted'] = (df['Chance of Admit'] >= 0.75).astype(int)

X = df[['GRE Score', 'CGPA']]
y = df['Admitted']
```

```
print(X.head())
print(y.head)
```

```
   GRE Score  CGPA
0        337  9.65
1        324  8.87
2        316  8.00
3        322  8.67
4        314  8.21
<bound method NDFrame.head of 0      1
1      1
2      0
3      1
4      0
      ..
495    1
496    1
497    1
498    0
499    1
Name: Admitted, Length: 500, dtype: int64>
```

```
# Train-test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
```

```
# Build Decision Tree model
model = DecisionTreeClassifier(random_state=42, max_depth=3)
model.fit(X_train, y_train)
```

```
DecisionTreeClassifier
DecisionTreeClassifier(max_depth=3, random_state=42)
```

```
# Prediction
y_pred = model.predict(X_test)
```

```
print("\nAccuracy:", accuracy_score(y_test, y_pred))
```

```
Accuracy: 0.91
```

```
print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred))
```

```
Confusion Matrix:
[[51  5]
 [ 4 40]]
```

```
print("\nClassification Report:")
print(classification_report(y_test, y_pred))
```

```
Classification Report:
              precision    recall  f1-score   support

           0       0.93      0.91      0.92        56
           1       0.89      0.91      0.90        44

    accuracy                           0.91       100
   macro avg       0.91      0.91      0.91       100
weighted avg       0.91      0.91      0.91       100
```